

Hi,

Welcome to the data cleansing homework. After the previous modules dedicated to the pure technical data ingestion, it's time to prepare our data for further usage. The preparation consists of various things, including fixing identified data quality issues or extending the incoming data with external sources.

Part 1 - data cleansing job

Prepare a data cleansing job. To simplify the implementation, the job composition is divided into the following 4 exercises.

Exercise 1 - data classification

Implement an Apache Spark Structured Streaming job that will read the *raw_data* Kafka topic. The job should write the incoming records to the following topics:

- *valid_data* if the event is fully valid
- *invalid_data* if the event is invalid; an event is considered invalid when one of the following is true:
 - *user_id* field is equal to 0
 - *visit_id* field is NULL

The Apache Kafka configuration is:

- `kafka.bootstrap.servers`: localhost:29092
- `subscribe`: *raw_data*
- `startingOffsets`: EARLIEST

Use the same configuration across all exercises.

Exercise 2 - data deduplication

Add a data deduplication code for the job from the previous exercise. The deduplication logic should be based on the *visit_id* and *event_time* attributes of the incoming events.

Exercise 3 - data enrichment

Combine the data cleansing job you've created in the previous exercises with an external database storing the information about devices. Below are the connection parameters for this PostgreSQL database:

- url: jdbc:postgresql:bde
- table: bde_schema.devices
- user: bde_user
- password: bde_password

The output written to the Kafka topic must include all fields present in the raw event plus ideally the `full_name` value coming from the enrichment dataset (= it can be missing). You can define any schema you want but it must contain these 2 values at this moment.

Exercise 4 - data preparation

Finally, add some data preparation logic to the job you're creating. There are some identified issues with the data you're processing. Fix them in the job before writing the events to *valid_data* or *invalid_data* topics. The identified issues are:

- technical.device field
 - can be missing
 - the expected structure is `{"type": "string", "version": "string"}` but can be `{"type": {"name": "string"}, "version": string}}`
 - version can be missing if the type is "pc"
- technical.network field
 - can be missing
 - the expected type is string but can be a structure `{"short_name": "string", "long_name": "string"}`
- technical.browser field
 - can be missing
 - the expected type is string but can be a structure `{"name": "string", "language": "string"}`. In case of structure, the lang field under technical structure (`technical.lang`) is missing
- source field
 - can be missing
 - site attribute should start without www. prefix but sometimes it's prefixed with it; ie. expected is `mysite.com`, not expected www.mysite.com

Additionally, add the `full_name` device attribute to the `technical.device` field. For example, if the `full_name` is "abc" and the input `technical.device` is `{"name": "xyz", "type": "opr"}`, we should end up with the structure like `{"name": "xyz", "type": "opr", "full_name": "abc"}`

Setup instructions - common for all the exercises

Java/Scala

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/02-data-cleansing/scala> in IntelliJ.
2. Implement your job.
3. Start Docker containers. They'll generate all necessary data for the exercise:
cd docker/p1
docker-compose down --volumes; docker-compose up

Python

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/02-data-cleansing/python> in PyCharm.
2. Implement your job.
3. Start Docker containers. They'll generate all necessary data for the exercise:
cd docker/p1
docker-compose down --volumes; docker-compose up

 **If you are stuck**, please check the *Homework - instructions* document present in the same page. It contains some extra details on how to implement various parts of the homework.

Have fun and see you in the next module!

Bartosz.